

Why Long Running AI Agents Fail

What Dedalus Should Build Next

A product brief for the agent management layer on top of Dedalus Machines

Sarvesh Tamse · github.com/sarvesh2003dev · Independent research on multi step agent failures

Executive Summary

Dedalus has built the hardest part of the stack: fast, persistent, economically viable compute for agents. Machines start in under 50 ms, sleep for free, and preserve filesystem and runtime state. That solves the compute substrate.

What remains is the agent management layer. Long running agents still fail because reasoning state is ephemeral, recovery is manual, and observability is incomplete. The highest leverage next product is a first class **Agent Checkpoint API** that turns “the machine survives” into “the agent survives.”

This brief maps failure modes to product gaps, compares the landscape, proposes a concrete feature with success metrics, and outlines a simple three phase roadmap.

The Problem

Long running agents (roughly 30 minutes to multi hour runs) fail in ways that better models alone cannot fix.

- **Context rot.** Tool outputs, intermediate reasoning, and error traces fill the window. Original instructions erode. By step 20 of a 30 step task the agent has often drifted.
- **No durable agent state.** Crashes, rate limits, deploys, or context limits wipe progress. Restarting wastes tokens and leaves inconsistent side effects such as half written files or duplicate API calls.
- **Blocking tool calls.** Long code execution, migrations, or scraping can block the model for minutes. The agent either idles, polls wastefully, or abandons the task.
- **Ephemeral reasoning memory.** The filesystem can persist. The agent’s working memory (context, decision history, intermediate plans) does not.

These are infrastructure problems. They appear across coding agents, research agents, and multi step workflow agents.

Who

Who feels the pain

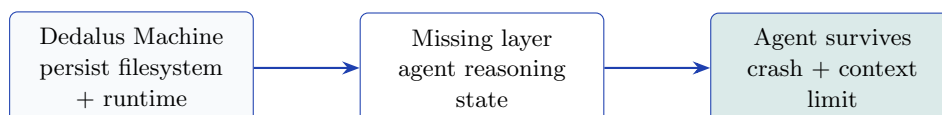
- Developers building autonomous coding, research, and data pipeline agents on Dedalus Machines.
- Frequency: any run longer than about 30 to 45 minutes.
- Cost of failure: 1 to 3 hours of developer time per failed run, wasted tokens, and manual cleanup of partial side effects. Teams that cannot trust long runs simply avoid building them.

What Dedalus Already Solves

Dedalus Machines provide a strong foundation:

- Full Linux VMs with persistent filesystem and runtime across sleep and wake.
- Sub 50 ms starts via snapshot restore and zero idle compute cost.
- Real execution environment (root, packages, Docker, GPU) so agents can run long processes.
- Unified API and MCP marketplace that remove tool configuration friction.

The machine survives. The gap is that the agent itself does not yet survive in a first class way.



Product Gaps

From public documentation and the current product surface, the following first class experiences are not yet prominently available:

- **Agent level checkpointing API.** No structured way to snapshot and restore context, tool call history, decision points, and intermediate outputs.
- **Execution observability for long runs.** Limited queryable timeline of tool calls, model outputs, state transitions, and errors when a multi hour run fails.
- **Managed context layer.** Developers must manually decide what stays in the context window versus what is archived to the filesystem.
- **Side effect tracking and rollback.** No simple “undo everything since step 15.”
- **Opinionated agent templates.** Common patterns still require assembly from scratch.

User Story

A developer launches a coding agent to refactor a mid sized repository. The agent spends 82 minutes exploring the codebase, writing tests, and applying changes. At minute 82 the connection drops or the context limit is hit.

Today: The developer starts over. Tokens are wasted. Partial file changes must be cleaned up by hand.

With a Checkpoint API: The platform had been checkpointing every N tool calls. The developer issues one restore call. The agent resumes from the last consistent state and finishes the remaining work in minutes instead of hours.

This is the difference between experimental agents and production grade infrastructure.

Competitive Landscape

Other systems address pieces of durability. None combine Dedalus style persistent VMs with first class agent reasoning checkpoints in the same product surface.

System	Strength	Gap relative to Dedalus	Implication
Temporal	Mature durable workflows, signals, retries	Not agent native; heavy for LLM tool loops	Great for business workflows, less natural for free form agents
LangGraph	Explicit graph state and checkpoints	Requires developer to design the graph; limited runtime isolation	Strong for structured agents, weak on raw compute persistence
Azure Durable Functions	Orchestrator pattern, replay	Cloud lock in; not built around full Linux agent sandboxes	Solid for event driven apps, not full agent machines
Trigger.dev	Background jobs, retries, observability	Task oriented rather than long lived agent reasoning	Good for async jobs, thinner on multi hour agent state
Dedalus (today)	Fast persistent VMs, free idle, real Linux, MCP	Agent reasoning state still mostly ephemeral	Best substrate; needs the management layer on top

Dedalus is uniquely positioned: it already owns the hard compute layer. Adding durable agent state would create a clear differentiation against pure workflow engines and pure graph frameworks.

Proposed Solution: Agent Checkpoint API

A first class API that snapshots an agent’s working state (context, tool call history, intermediate outputs, and relevant filesystem diff) at configurable intervals or after critical tool calls. On crash, context limit, or explicit request, restore is a single API call.

W

hy this feature first

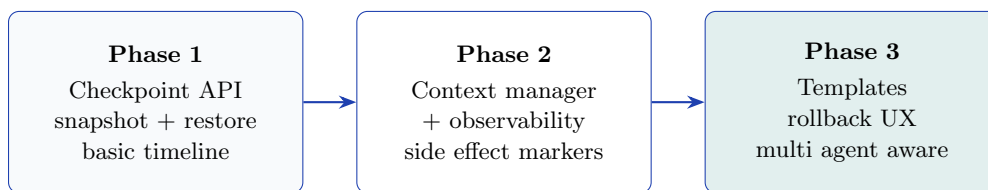
- It is the foundation. Observability needs durable state to observe. Context management needs something to archive. Rollback needs something to restore.
- Highest visible developer win. Resume in one call versus restart from zero is immediately felt.
- Leverages existing Dedalus strengths. The VM already persists the filesystem; the API extends persistence to the reasoning layer.
- Beats near term alternatives:
 - Better MCP integrations help, but agents still die mid run.
 - Multi agent orchestration multiplies the same durability problems.
 - Memory compression and prompt debugging assume the agent can survive long enough to use them.

S

uccess metrics

- Resume success rate after crash or context limit (target: greater than 95 percent of long runs can resume).
- Developer time saved per recovered run (hours of re execution avoided).
- Checkpoint restore latency (target: seconds, not minutes).
- Share of agent runs that exceed 30, 60, and 120 minutes (adoption signal that developers now trust long runs).
- Reduction in internal or support time spent on recovery scripts and “start over” incidents.

Simple Roadmap



- **Phase 1 (foundation):** Ship Checkpoint API and a minimal run timeline. Measure resume rate and restore latency.
- **Phase 2 (management):** Add managed context archival and richer observability. Introduce side effect markers for safer recovery.
- **Phase 3 (product surface):** Opinionated templates, one click rollback UX, and multi agent aware checkpoints.

Closing

Dedalus has already solved the compute problem that most agent platforms still struggle with. The next product surface is the agent management layer. A first class Checkpoint API is the highest leverage step that turns persistent machines into production grade long running agents.

I would be glad to discuss prioritization, metrics, or a more detailed PRD.

Sarvesh Tamse · sarvestamse08@gmail.com · +91 94213 00440 · github.com/sarvesh2003dev/ToolReliBench
Independent research on LLM reasoning failures across 15 to 30 step agentic tasks (GPT-4o, Claude, Gemini, Grok, Llama).